# Adaptive Meal Preparation in a Campus Canteen

A tabular Q-learning formulation for uncertain student demand

**TEAM MEMBER:** Shahid Samir Khatib - B24CM1055

**GITHUB:** https://github.com/Shahid20-K/AI-A2-Canteen-qlearning.git

## 1. Problem statement

A campus canteen must prepare enough meals to meet changing student demand without preparing so much that perishable food is discarded. Fixed preparation plans may respond poorly to day-to-day demand changes. This project studies whether a learned policy can balance availability and waste in a controlled simulation.

## 2. Environment and demand

One simulated day contains eight time slots. Nominal demand means are **(8, 12, 22, 30, 24, 18, 27, 14)** portions. A daily outlook is low, normal, or high, with multipliers 0.70, 1.00, and 1.30 and probabilities 0.25, 0.50, and 0.25. Actual demand is sampled from a Poisson distribution around the slot mean multiplied by the outlook factor.

The inventory capacity is 40 portions. Of unsold portions, 30% (rounded down) can carry into the next slot; the remainder is discarded. Nothing carries beyond the final slot. These are explicit synthetic assumptions, not measured canteen data.

At each step, available stock equals starting inventory plus actual preparation, capped at capacity. Sales are the minimum of available stock and demand; unmet demand is the difference when demand exceeds stock. Carryover and discarded waste are then calculated before the next slot.

## 3. State space S

The agent observes **(time slot, exact integer inventory, daily demand outlook)**. Inventory is represented exactly from 0 to 40 portions. In this optimistic simulation, the observed low/normal/high outlook identifies the day's demand regime; slot demand remains a random Poisson draw. Real forecasts are not perfectly accurate.

**State count = 8 × 41 × 3 = 984 discrete states.**

This compact tabular representation is intentionally chosen so that the learned values can be inspected and explained in a viva. The state includes the current time and the information available to the simulated decision-maker; demand remains uncertain.

## 4. Action space A

At each slot, the agent selects an additional preparation batch from **{0, 5, 10, 15, 20}** portions. The 20-portion per-slot maximum is an assumed batch-size limit, not a measured kitchen capacity. Separately, total inventory is capped at 40 portions.

## 5. Episode objective and hypothesis

One episode is a complete simulated day of eight decisions; its return is the sum of slot rewards. The experiment tests whether a state-dependent learned policy can outperform static preparation rules when the daily demand outlook changes. The policy is not assumed to succeed: this claim is assessed using held-out simulated episodes and operational metrics.

## 6. Slot transition

For slot t, available stock is $A_t = \min(I_t + p_t, 40)$. Sales are $\min(A_t, D_t)$ and unmet demand is $\max(D_t - A_t, 0)$. Leftover stock is $\max(A_t - D_t, 0)$; carryover is floor(0.30 × leftover) except at the final slot, where it is zero. Discarded waste equals leftover minus carryover.

Exact inventory avoids merging materially different stock levels into one state. The observed outlook identifies the demand regime in this simplified simulation, while the exact demand draw remains random. This makes the specified environment a finite-state decision process, under the stated assumptions.

## 7. Reward function R

The reward encourages serving demand while accounting for preparation, stockouts, and food discarded:

$$r = \text{sold} - 0.35 \times \text{prepared} - 1.00 \times \text{unmet demand} - 1.50 \times \text{discarded waste}$$

- **sold:** portions served to students.
- **prepared:** additional portions actually produced in the slot.
- **unmet demand:** demand not met because inventory was insufficient.
- **discarded waste:** unsold portions that do not carry over.

The coefficients are illustrative utility weights, not monetary estimates. They make competing objectives explicit. Changing these weights can change the policy, so they should be reported and justified rather than tuned invisibly.

## 8. Learning algorithm

We use tabular Q-learning with epsilon-greedy exploration. The update is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

At a terminal transition, the future-value term is zero. Each agent trains for 20,000 episodes with learning rate $\alpha = 0.12$, discount factor $\gamma = 0.92$, initial $\varepsilon = 1.0$, minimum $\varepsilon = 0.05$, and multiplicative decay 0.9992. The main stored agent is seed 2026 and acts greedily at test time; five training seeds assess seed sensitivity.

Early in training, epsilon-greedy exploration tries different preparation batches; as epsilon decays, the agent increasingly uses actions with the highest current Q-value. Discounting values rewards that occur later in the simulated day, so a decision can account for carryover into the next slot.

## 9. Evaluation design

Each policy is tested on the same 2,000 held-out simulated days using matched episode seeds. A forecast-aware rule's target fraction is selected from 50%, 60%, 70%, 80%, 90%, 100%, 110%, 120% using a separate 1,000-day validation set; the final test set is not used for tuning. Five independently trained agents assess seed sensitivity. Paired reward differences, confidence intervals, and 0.75×/1.25× demand-shift tests are recorded.

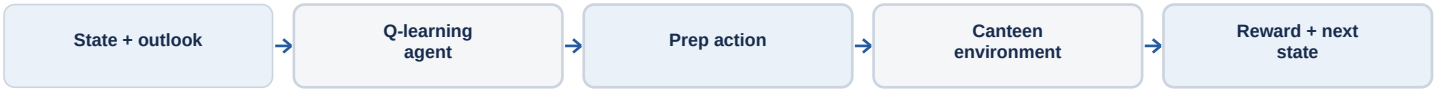
## 10. Reproducibility

The primary agent uses training seed 2026; four more agents use independent seeds. Test episodes are separate from training and use matched demand draws across policies. Outputs include the Q-table, returns for all seeds, metrics JSON, plots, a policy heatmap, and an eight-slot example trace. Thirteen unit tests cover state encoding/decoding, transition and reward invariants, configuration validation, training outputs, and reproducible matched-seed evaluation.

## 11. Scope of evidence

The baseline comparison supports conclusions only about this simulation and reward design. It does not establish that the same improvement would occur in a live canteen. Real deployment would require calibrated demand data, cost estimates, and operational and food-safety constraints.

### RL decision cycle




*The next state and reward feed into the Q-value update; the loop repeats until the simulated day ends.*

# Experimental Results and Discussion

Synthetic simulation results, interpretation, and limitations

Synthetic simulation only; no real canteen records used.

Main test: 2,000 held-out synthetic days per policy; Q-learning is the seed-2026 trained agent. The agent was also trained with five independent seeds. All results are outcomes from this specified simulation, not observations from a real canteen.

| Policy | Mean reward/day | Sold/day | Unmet demand/day | Waste/day |
|---|---|---|---|---|
| Q-learning (seed 2026) | 30.43 | 120.90 | 34.58 | 7.34 |
| Fixed 15-portion batch | -5.06 | 106.41 | 49.07 | 13.59 |
| Forecast-aware tuned schedule | 30.76 | 122.60 | 32.88 | 8.67 |

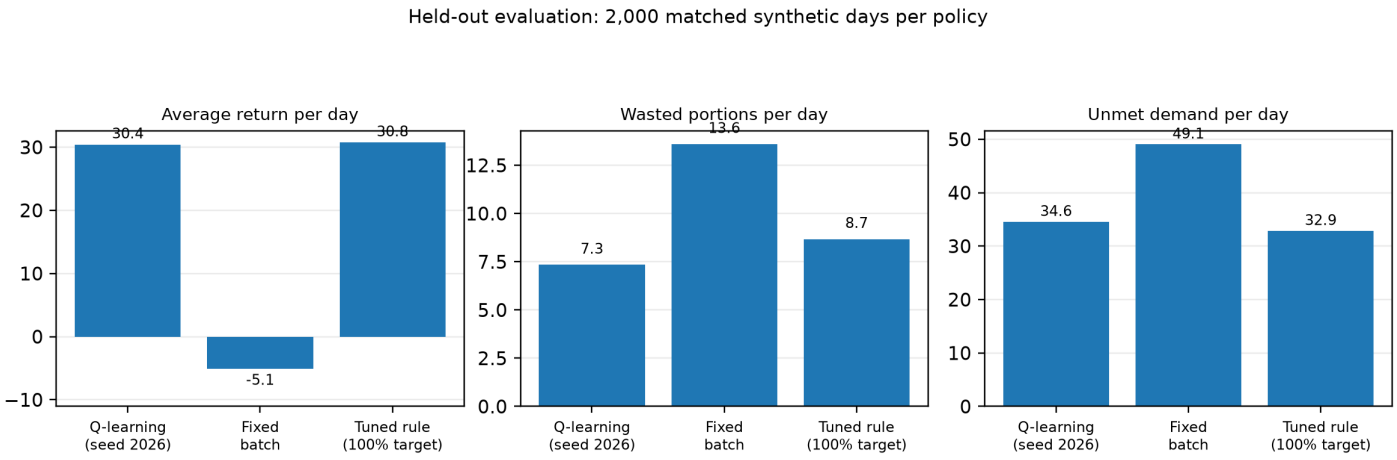



*Figure 1. Mean reward, waste, and unmet demand for the learned policy and comparison baselines.*

## 12. Learning behavior

The curve plots 200-episode return means averaged across five training seeds, with a ±1 SD band. Returns rise early and then fluctuate around a stable range. This is evidence of training progress in the simulator, not proof of real-world performance.

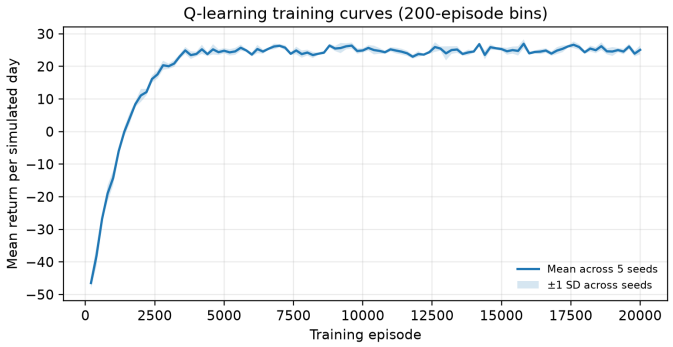



*Figure 2. Mean return in 200-episode bins across five seeds; shaded band is ±1 SD.*

## 13. Interpretation and paired checks

On the held-out test set, Q-learning reward was 30.43 (95% CI 29.79 to 31.08) versus 30.76 for the tuned schedule (95% CI 30.10 to 31.42). The paired reward difference (Q-learning minus baseline) was -0.33 (95% CI -0.56 to -0.10); versus fixed-batch it was 35.49 (34.97 to 36.00). Q-learning discarded fewer portions (7.34 vs 8.67) but sold fewer and had more unmet demand than the tuned schedule. Across five training seeds, mean Q-learning reward was 30.12 (SD 0.80).

Under demand shifts, mean reward for Q-learning versus the nominal-planning tuned schedule was 20.86 vs 16.30 at 0.75× demand, and 8.75 vs 11.58 at 1.25×. Both policies keep their nominal planning assumptions; the results show sensitivity to a changed environment.

## 14. Limitations

Demand is synthetic and Poisson-distributed. The environment assumes one meal type, eight time slots, a perfect categorical regime outlook, a 20-portion maximum action, simplified carryover, and illustrative reward weights. The forecast-aware baseline slightly exceeds the learned agent on nominal reward; Q-learning trades some service for lower waste. No real-world savings are established.

## 15. Reward-weight sensitivity

| Train waste wt. | Base reward/day | Waste/day | Unmet/day |
|---|---|---|---|
| 0.75 | 29.62 +/- 0.04 | 9.05 +/- 0.30 | 33.28 +/- 0.35 |
| 1.50 | 28.53 +/- 1.03 | 6.45 +/- 1.57 | 36.85 +/- 2.39 |
| 2.25 | 27.49 +/- 0.11 | 4.01 +/- 0.03 | 40.22 +/- 0.10 |

Each setting trains three seeds; each policy is tested on the same 2,000 held-out scenarios using the original reward weights. Values are mean +/- SD across trained seeds, not confidence intervals. As the training waste penalty rises, waste falls but unmet demand rises. This demonstrates reward sensitivity, not an objectively optimal coefficient.

## 16. Future work

Next steps are to test imperfect forecasts, calibrate assumptions with permitted anonymized canteen records, and model freshness, preparation lead times, and food-safety constraints.

## Conclusion

Within this synthetic simulation, Q-learning greatly improves over the fixed 15-portion batch baseline, but the validation-tuned forecast-aware rule slightly exceeds it on nominal mean reward and has fewer stockouts. Q-learning produces less waste than that tuned rule. No single policy wins on every metric, and results do not establish measured campus-wide impact.

Reproducibility: source, trained Q-table, tests, metrics, sensitivity analysis, and figure scripts are in the repository once the GitHub placeholder is replaced.